

Continuous deployment (CD) in GitLab

Production environments receive the automatic release of code changes following successful CI testing.

GitLab's CD capabilities include:

Environment management:

GitLab streamlines the administration of diverse ecosystems, including development, testing, and deployment settings, thus enabling maintenance of standardised setups.

Auto DevOps: By employing best practices, GitLab's Auto DevOps function sets up CI/CD pipelines swiftly and with minimal effort.

Paradigm shift towards Auto DevOps

Auto DevOps boasts an AI-assisted functionality that scrutinises the codebase and establishes automatic CI/CD pipeline configurations. By integrating past data and machine learning models, Auto DevOps streamlines pipeline phases. This encompasses choosing the right runner types and tuning parameters depending on the project's properties. Individualised CI/CD pipeline configurations are established using this method, eliminating the need for manual tweaking.

The Auto Remediation tool is intended to automatically repair failed CI/CD projects. Using AI insights, Auto Remediation provides customised advice or job retry suggestions, saving developers time and effort. This feature reduces the manual intervention required to address common build or test failures, promoting faster delivery of code changes.

GitLab's platform offers extensive cybersecurity options for CI/CD pipelines. It prioritises cybersecurity-enabled features by incorporating robust security measures into its CI/CD platform, allowing developers to detect and mitigate security risks during development.

SAST vs DAST testing in GitLab AI services

The purpose of a static application security testing (SAST) tool is to analyse the source code of software applications to identify potential security vulnerabilities and coding errors. SAST is a type of white-box testing — it examines the internal structure and logic of the code without executing the application. The primary goal of SAST is to proactively detect security flaws early in the software development life cycle, allowing developers to address and fix these issues before the application is deployed or released to production. GitLab's SAST tool performs a detailed review of the source code, locating security weaknesses like code injection, incorrect authentication, and XSS threats. SAST integration within the CI/CD pipeline helps ensure a secure software release by identifying and addressing security concerns earlier in development.

Dynamic application security testing (DAST) is a type of security testing that assesses web applications and APIs while they are running. Unlike SAST, which examines the source code of an application, DAST operates from the outside-in, simulating real-world attacks by interacting with the application through its front-end, just like an external user would. The running state of web applications is checked by GitLab's DAST security feature in their CI/CD platform. Using DAST to scan deployed apps reveals potential flaws that may escape detection through other methods. An anticipatory security tactic offers robust defence against prospective dangers. There are two forms of DAST testing in GitLab, which use AI features to automate test management.

Dependency scanning: Automated dependency scanning in GitLab identifies recognised security weaknesses among project dependencies like libraries

and packages. Dependency scanning integration within the pipeline minimises the chance of threats exploitation by proactively identifying and flagging obsolete or unsafe components.

Container scanning:

Containerisation being more widely adopted has led GitLab to include container scanning in its CI/CD workflow. The feature flags any security concerns found within container images, including antiquated packages or an inadequate setup. Safeguarding containers strengthens comprehensive application security during deployment.

Shifting from DevOps adoption to DevSecOps adoption

DevOps is termed as a 'cultural movement' that emphasises the collaboration and communication between development, operations, and security teams. The goal of DevOps is to shorten the software development life cycle and improve the quality of software by automating the delivery process.

DevSecOps is an extension of DevOps that brings security into the fold. It ensures that security is considered at every stage of the software development life cycle, from the initial design phase to the final deployment.

Here are some of the key differences between DevOps adoption and DevSecOps adoption:

- DevOps focuses on speed and efficiency, while DevSecOps focuses on security. DevOps teams are often focused on getting code out the door quickly, while DevSecOps teams are focused on making sure that code is secure before it is deployed.
- DevOps teams are often siloed, while DevSecOps teams are more collaborative. DevOps teams are often made up of developers and operations engineers, while DevSecOps teams also include security engineers. This collaboration helps to ensure that security is considered at